

CO4-AT1- CODING CHALLENGE

CSA0603-Design and Analysis of Algorithms (DAA)

Name: VANGIMALLA BALA SIVA PRASAD REDDY

Register Number: 192424316

Q39. Decode Ways Problem

A string of digits represents encoded letters where '1' to '26' map to 'A' to 'Z'. The task is to count how many ways the string can be decoded. The goal is to determine all valid decoding combinations. Use Dynamic Programming to compute efficiently.

Aim

To design and implement a Dynamic Programming algorithm to determine the number of possible ways to decode a string of digits, where digits from 1 to 26 represent the letters A to Z.

Problem Statement

A string of digits represents encoded letters using the following mapping:

1 → A

2 → B

3 → C

...

26 → Z

Given a string of digits, the objective is to find the total number of valid ways in which the string can be decoded.

For example, the input 226 can be decoded as:

2 2 6 $\rightarrow$ BBF

22 6 $\rightarrow$ VF

2 26 $\rightarrow$ BZ

Therefore, the total number of ways is 3.

Objective

- To understand the Decode Ways problem.
- To apply Dynamic Programming to solve the problem efficiently.
- To calculate the total number of valid decoding possibilities.
- To generate all possible valid decoding combinations.
- To analyze the time and space complexity of the algorithm.

Algorithm

1. Read the input string s.
2. Check whether the string is empty or starts with 0.
3. Create a DP array dp, where dp[i] represents the number of ways to decode the first i characters.
4. Initialize: dp[0] = 1 and dp[1] = 1 if the first digit is non-zero.
5. For every position from 2 to n:
6. Consider the current digit as a single character.
7. If it is not 0, add dp[i-1] to dp[i].
8. Consider the last two digits.
9. If the two-digit number lies between 10 and 26, add dp[i-2] to dp[i].

10. Store valid decoding combinations while performing the DP computation.
11. The final value $dp[n]$ gives the total number of decoding ways.
12. Display the number of ways and all valid combinations.

Pseudo code

START

Read string s

$n \leftarrow \text{length}(s)$

IF s is empty OR first character is '0'

 Print 0

 STOP

END IF

Create $dp[0\dots n]$

Create combinations[0...n]

$dp[0] \leftarrow 1$

combinations[0] $\leftarrow \{""\}$

$dp[1] \leftarrow 1$

combinations[1] $\leftarrow$ character corresponding to $s[0]$

FOR $i \leftarrow 2$ TO n

 IF $s[i-1]$ is not '0'

$dp[i] \leftarrow dp[i] + dp[i-1]$

```

    FOR each decoding in combinations[i-1]
        Append current character
        Store in combinations[i]
    END FOR
END IF

twoDigit ← integer value of s[i-2...i-1]

IF twoDigit >= 10 AND twoDigit <= 26
    dp[i] ← dp[i] + dp[i-2]

    FOR each decoding in combinations[i-2]
        Append character corresponding to twoDigit
        Store in combinations[i]
    END FOR
END IF

END FOR

Print dp[n]
Print all decoding combinations

STOP

```

Python Program

```
class Solution:
```

```
    def decodeWays(self, s):
```

```
        n = len(s)
```

```
        if n == 0 or s[0] == '0':
```

```
return 0, []
```

```
dp = [0] * (n + 1)
```

```
combinations = [[] for _ in range(n + 1)]
```

```
dp[0] = 1
```

```
combinations[0] = [""]
```

```
dp[1] = 1
```

```
combinations[1] = [chr(64 + int(s[0]))]
```

```
for i in range(2, n + 1):
```

```
    if s[i - 1] != '0':
```

```
        dp[i] += dp[i - 1]
```

```
        for value in combinations[i - 1]:
```

```
            combinations[i].append(
```

```
                value + chr(64 + int(s[i - 1]))
```

```
            )
```

```
two_digit = int(s[i - 2:i])
```

```
if 10 <= two_digit <= 26:
```

```
    dp[i] += dp[i - 2]
```

```
    for value in combinations[i - 2]:
```

```
        combinations[i].append(
```

```

        value + chr(64 + two_digit)

    )

    return dp[n], combinations

s = input("Enter the string: ")

ways, combinations = Solution().decodeWays(s)

print("\nDP Table")

print("-----")

print("i\tSubstring\tdp[i]\tDecoding")

print("-----")

for i in range(len(s) + 1):

    if i == 0:

        substring = "Empty"

        decoding = "—"

    else:

        substring = s[:i]

        if combinations[i]:

            decoding = ", ".join(combinations[i])

        else:

            decoding = "—"

    print(f"{i}\t{substring:<10}\t{len(combinations[i])}\t{decoding}")

```

```
print("-----")
```

```
print("Ways =", ways)
```

Sample Input

226

Sample Output

Ways = 3

Decoding Combinations:

1 -> BBF

2 -> BZ

3 -> VF

Output

```
Enter the string: 226
Ways = 3
Decoding Combinations:
1 -> BBF
2 -> VF
3 -> BZ
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

Enter the string: 226

DP Table

i	Substring	dp[i]	Decoding
0	Empty	1	—
1	2	1	B
2	22	2	BB, V
3	226	3	BBF, VF, BZ

Ways = 3

Manual Calculation

For the input:

226

Step 1: First digit

2 $\rightarrow$ B

So:

dp[1] = 1

Step 2: First two digits

The possibilities are:

2 2 $\rightarrow$ BB

22 $\rightarrow$ V

Therefore:

$$dp[2] = 2$$

Step 3: Three digits

For the last digit 6:

$$2\ 2\ 6 \rightarrow \text{BBF}$$

For the last two digits 26:

$$2\ 26 \rightarrow \text{BZ}$$

Also, using 22 followed by 6:

$$22\ 6 \rightarrow \text{VF}$$

Therefore:

$$\begin{aligned} dp[3] &= dp[2] + dp[1] \\ &= 2 + 1 \\ &= 3 \end{aligned}$$

DP Table

i	Substring	dp[i]	Decoding
0	Empty	1	—
1	2	1	B
2	22	2	BB, V
3	226	3	BBF, BZ, VF

Result

The Dynamic Programming algorithm successfully calculates the number of valid ways to decode the given digit string.

For the input 226, the valid decoding combinations are:

BBF, BZ, VF

Hence,

Number of Ways = 3

Complexity Analysis

Time Complexity: $O(n + k)$

where n is the length of the input string and k is the number of decoding combinations generated.

Space Complexity: $O(n + k)$

The DP array requires $O(n)$ space, while storing all decoding combinations requires additional space depending on the number of valid combinations.

Conclusion

The Decode Ways problem was successfully solved using the Dynamic Programming approach. The algorithm efficiently determines the number of valid ways to decode a given string by considering both single-digit and two-digit mappings. For the input 226, the algorithm identified three valid decoding combinations: BBF, BZ, **and** VF. Hence, the total number of decoding ways is 3. The approach avoids unnecessary repeated calculations and provides an efficient solution while also generating all possible valid decoding combinations.